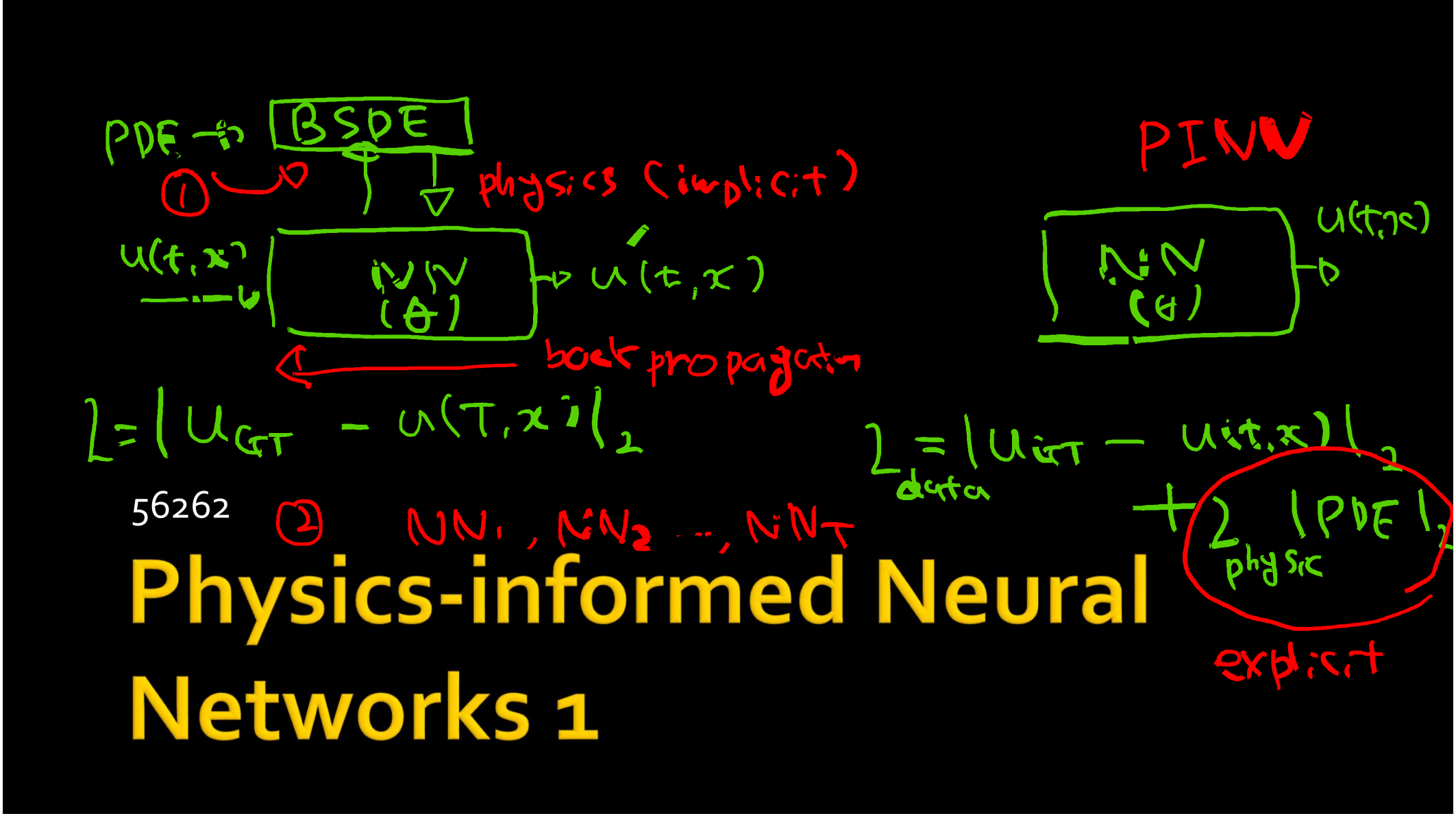
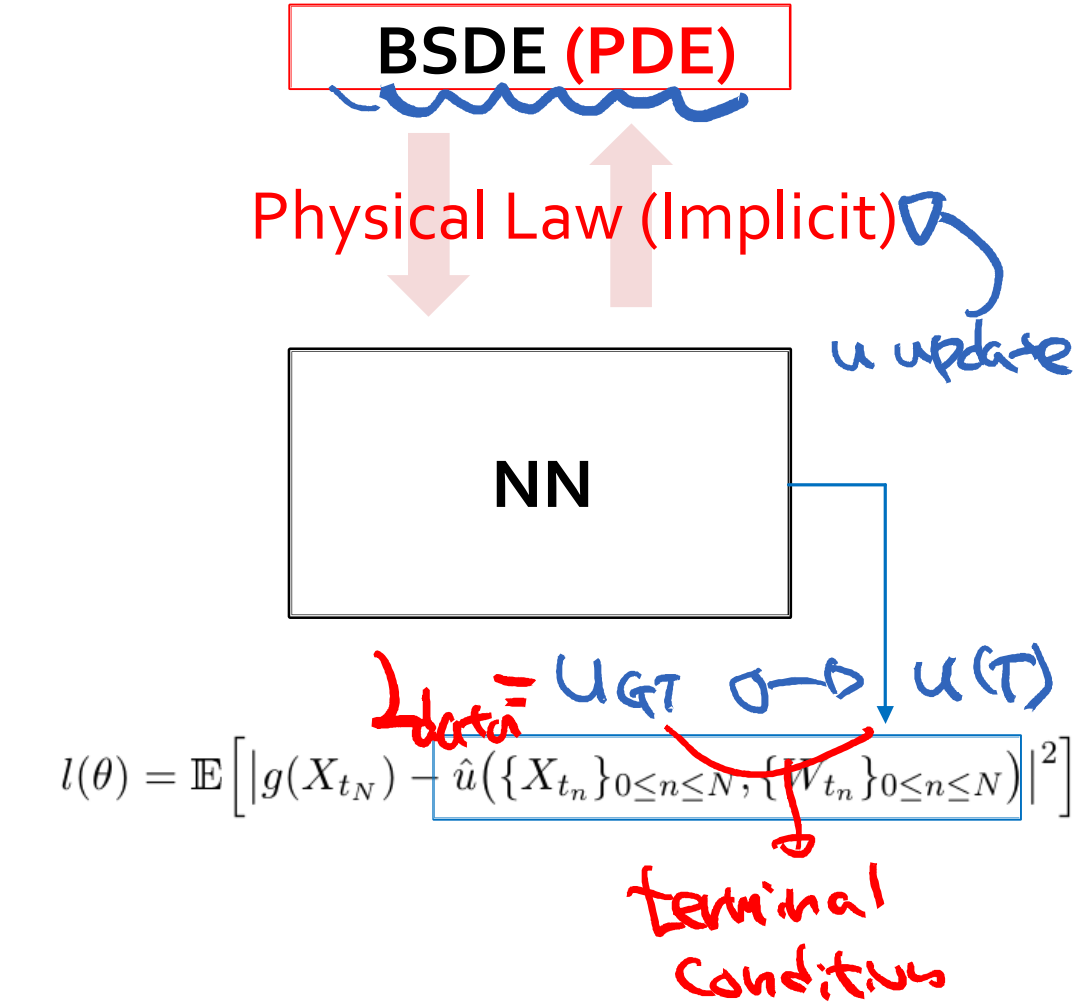
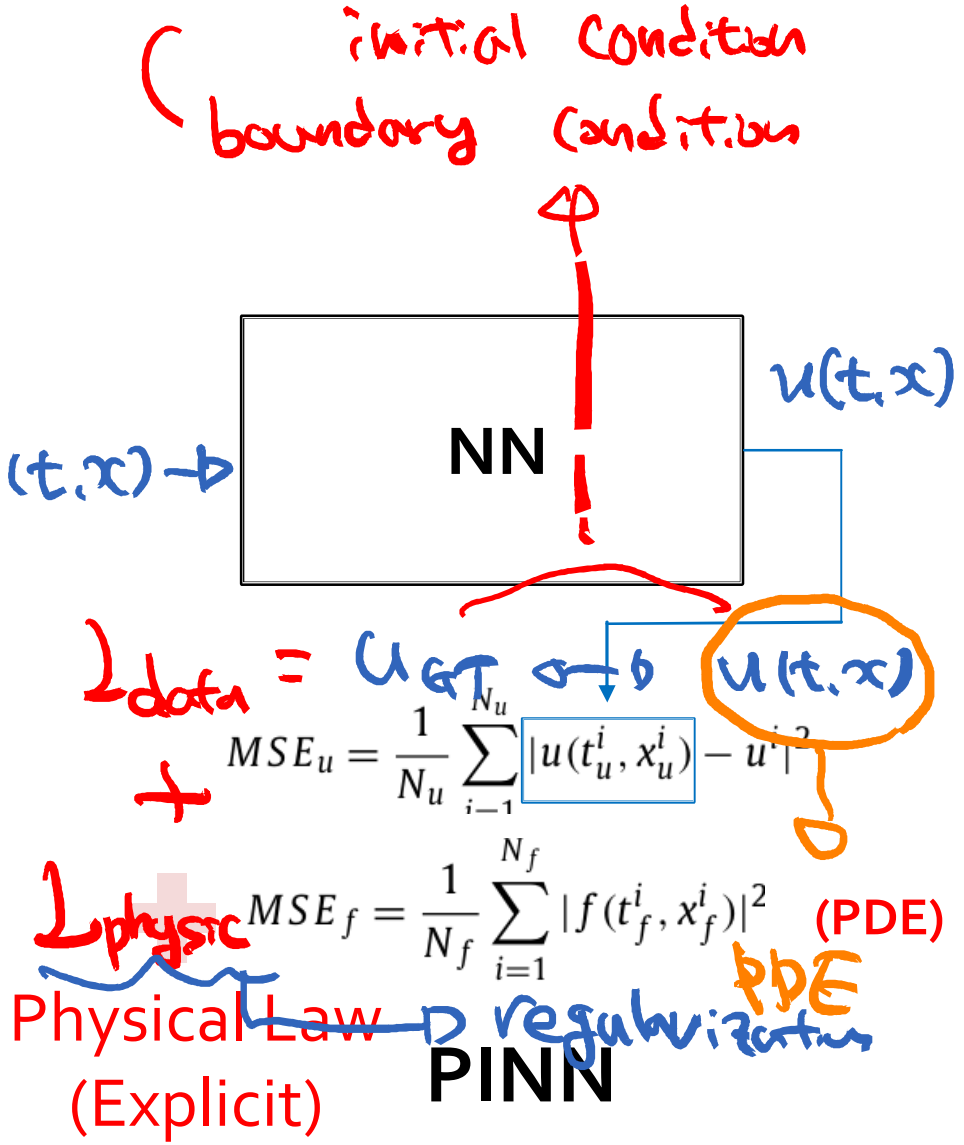


Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations
(*Journal of Computational Physics, 2019*)

PDE using DL vs. PINN



PDE using DL



PINN

Problem Statement 1/2

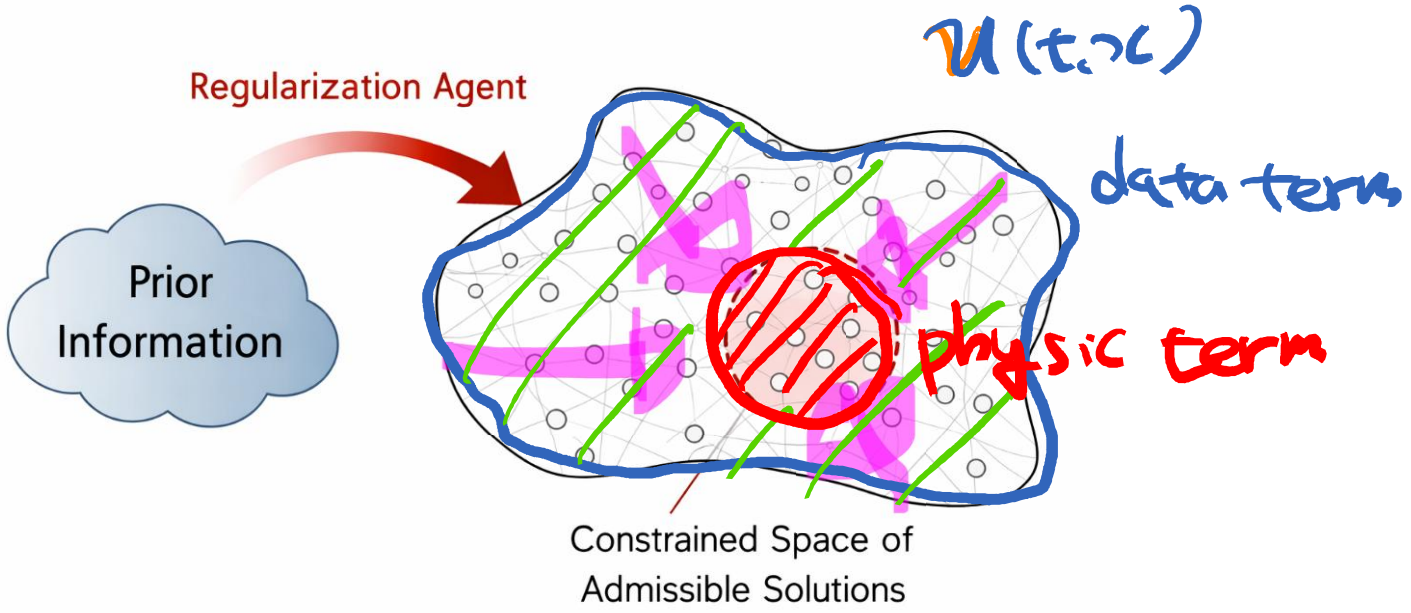
- **Physics-informed Neural Networks (PINN)**
  - Neural networks that are trained to solve supervised learning tasks while respecting any given laws of physics described by general nonlinear partial differential equations.

physic law

- **Strong Inductive Bias**
  - There exists a vast amount of prior knowledge that is currently not being utilized in modern machine learning.
  - This prior information can act as a regularization agent that constrains the space of admissible solutions to a manageable size

Problem Statement 2/2

- **Strong Inductive Bias**
  - It enables to quickly steer itself towards the right solution and generalize well even when only a few training examples are available.



Problem Setup 1/3

- **Nonlinear Partial Differential Equations**

$$u_t + \mathcal{N}[u;\lambda] = 0, \quad x \in \Omega, \quad t \in [0,T],$$

- $u(t,x)$: unknown solution
- $\mathcal{N}[\cdot]$: nonlinear differential operator
- Ω: spatial domain
- $[0,T]$: time interval

Condition

ex)

$$\mathcal{N}[u;\lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$$

- **Goal**
  - Given fixed model parameters λ, it aims to find the unknown hidden state $u(t,x)$ of the system.

Assumption

Problem Setup 2/3

- **Initial and Boundary Conditions :**To make the problem well-posed, the PDE is accompanied by:
  1. Initial condition

$$u(0, x) = u_0(x)$$

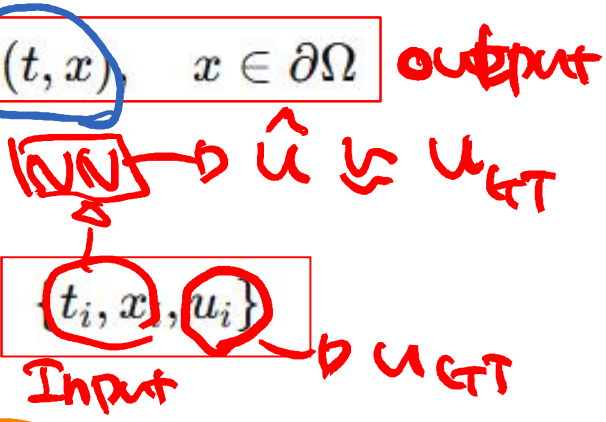
  2. Boundary condition

$$u(t, x) = g(t, x), \quad x \in \partial\Omega$$

- **Training Data**
  - A limited set of observations

$$\{t_i, x_i, u_i\}$$

  - These can come from initial/boundary conditions



Problem Setup 3/3

- **Objective**: The objective is to find a function $u(t, x)$ such that
  1. It respects initial and boundary conditions (observed data)
  2. It satisfies the governing PDE

- **Key Idea**: Instead of solving the PDE numerically
  - Represent $u(t, x)$ using a neural network

$$u(t, x) \approx u_\theta(t, x)$$

  - Enforce the PDE as a constraint during training

PINN 1/4

- **PDE**

$$u_t + \mathcal{N}[u] = 0$$



- **PINN**
  - PDE to Residual Form

$$f(t,x) := u_t + \mathcal{N}[u]$$

$\hat{u}_t + N[u_t] = f(t,x) \to 0$ (residual $\to 0$)

  - The objective is to enforce $f(t,x) = 0$
  - Instead of solving the PDE directly, the model is trained to minimize this residual.

PINN 2/4

- **Loss Function**
  - The total loss consists of two components:

$$\mathcal{L} = \mathcal{L}_u + \mathcal{L}_f$$

Data Loss

Physics Loss

fidelity term

regularization term

- **Data Loss**
  - Enforces initial and boundary conditions

$$\mathcal{L}_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2,$$

$|u_{GT} - \hat{u}|$

initial condition

boundary condition

- **Physics Loss**
  - Forces the model to satisfy the PDE

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

u_0

PDE

PINN 3/4

- **Pipeline**
  1. Input $(t, x) \rightarrow \mathrm{NN}_\theta \rightarrow$ Output $u(t, x)$
  2. Training data
     - Initial and Boundary Conditions (**Supervised**)

$$\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$$

labels

Input Output

$x \in \Omega$
$t \in [0, T]$

$L_{physics} |f(x,t)|_2$

     - Physics constrains (**Unsupervised**)

$$\{t_f^i, x_f^i\}_{i=1}^{N_f}$$

No labels

PDE

The residual is evaluated at sampled points:
- These are called **collocation points**
- No labeled data is required at these points

This enables learning even with **limited supervision**

PINN 4/4

- **Pipeline**
  3. Compute derivatives via automatic differentiation
     - Automatic Differentiation (AD) is a technique used to **compute exact derivatives of functions defined by code.**
       - $u_t = \text{grad}(u, t)$
       - $u_x = \text{grad}(u, x)$
       - $u_{xx} = \text{grad}(u_x, x)$
  4. Evaluate PDE residual $f(t, x)$
  5. Compute loss $\mathcal{L} = \mathcal{L}_u + \mathcal{L}_f$
  6. Update parameters θ via Backpropagation

Example 1/6

- **Schrödinger Equation**
  - How the state of a system evolves over time in the quantum world.

$$ih_t + 0.5h_{xx} + |h|^2 h = 0$$

    - $h(t, x)$: complex-valued wave function
    - i: imaginary unit

  - **Goal :** learn $h(t, x)$ over space and time

$$h(t, x) = u(t, x) + iv(t, x)$$

    - $u(t, x)$: real part
    - $v(t, x)$: imaginary part

Example 2/6

- **Schrödinger Equation**
  - **Neural Network**
    - Input: (t, x)
    - Output: $u(t, x), v(t, x)$
  - **PDE Reformulation :** Both residuals must be close to zero
    - Real part:

$$0 \approx f_u = u_t + 0.5 v_{xx} + (u^2 + v^2) v$$

    - Imaginary part:

$$0 \approx f_v = v_t - 0.5 u_{xx} - (u^2 + v^2) u$$

Example 3/6

- **Schrödinger Equation**
  - **Automatic Differentiation :** Derivatives are computed using AD (u_t, u_{xx} v_t, v_{xx})
  - **Loss Function :** Data term

$$x \in [-5, 5], \quad t \in [0, \pi/2]$$
$$h(0, x) = 2\,\mathrm{sech}(x)$$
$$h(t, -5) = h(t, 5),$$
$$h_x(t, -5) = h_x(t, 5),$$

Initial Condition

$$MSE_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |h(0, x_0^i) - h_0^i|^2,$$

$$MSE_b = \frac{1}{N_b} \sum \left(|h^i(t_b^i, -5) - h^i(t_b^i, 5)|^2 + |h_x^i(t_b^i, -5) - h_x^i(t_b^i, 5)|^2 \right),$$

Boundary Condition

Example 4/6

- **Schrödinger Equation**
  - **Loss Function :** Physics term

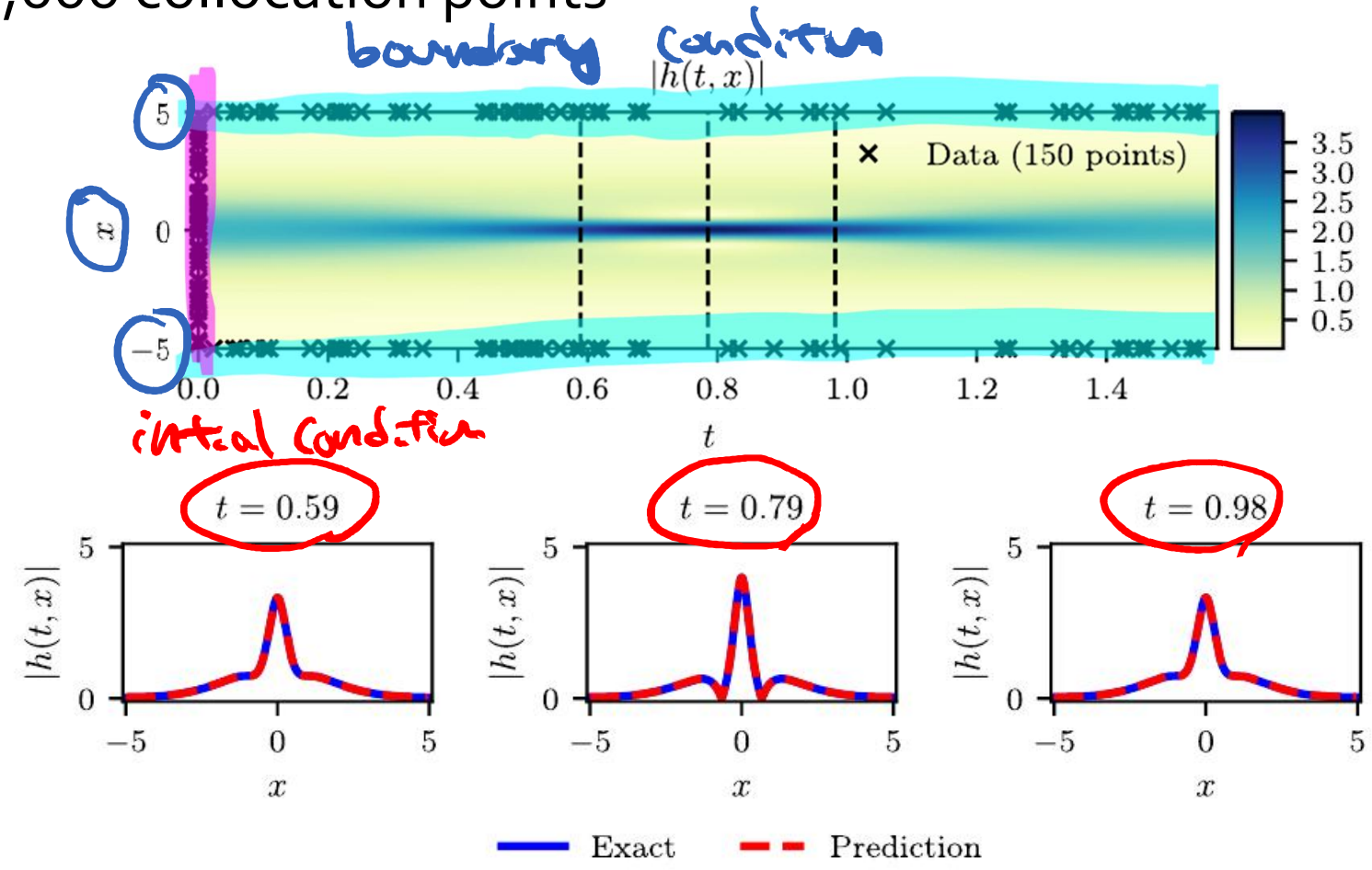
$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

  - **Total Loss Function :** Data term + Physics term

$$MSE = MSE_0 + MSE_b + MSE_f,$$

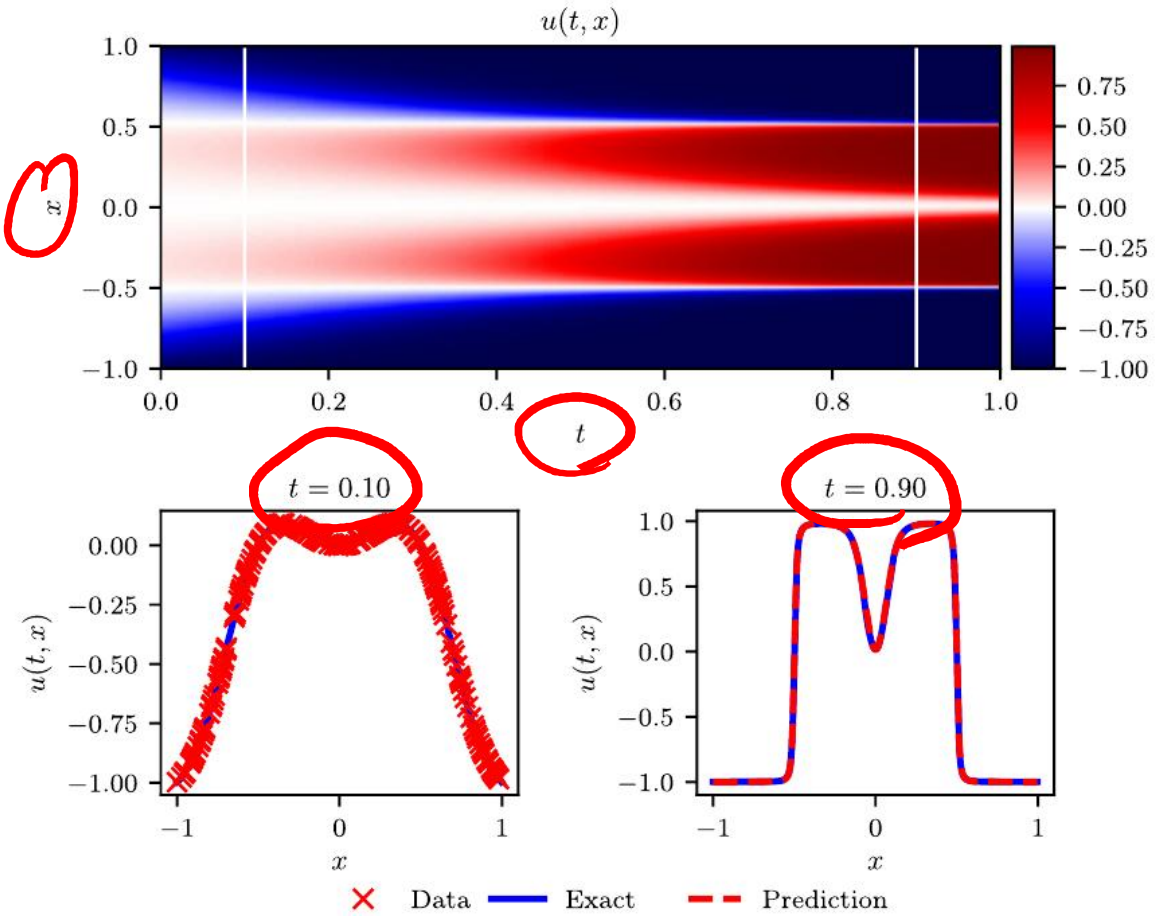
Example 5/6

- **Schrödinger Equation :** Predicted solution $|h(t,x)|$ along with the initial and boundary training data. We are using 20,000 collocation points



Example 6/6

- **Allen–Cahn equation :** solution $u(t, x)$ along with the location of the initial training snapshot at $t = 0.1$ and the final prediction snapshot at $t = 0.9$.



Data-driven Discovery of PDE 1/2

- **Problem Definition**

$$u_t + \mathcal{N}[u; \lambda] = 0$$

- $u(t, x)$: unknown solution (observed through data)
- $\mathcal{N}$: nonlinear differential operator
- λ: ~~Fixed~~ → **unknown parameters to be discovered**

- **Goal**
  - Given observed data (possibly noisy)

  → **identify the governing PDE**, i.e., estimate λ

Data-driven Discovery of PDE 2/2

- **Core Idea :** The PDE parameters are learned together with the neural network
  - Step 1: Approximate the solution

$$u(t,x) \approx \mathrm{NN}(t,x)$$

  - Step 2: Define the PDE residual

$$f = u_t + \mathcal{N}[u;\lambda]$$

    - The parameters λ become **trainable variables**

$$L = L_{data} + L_f$$

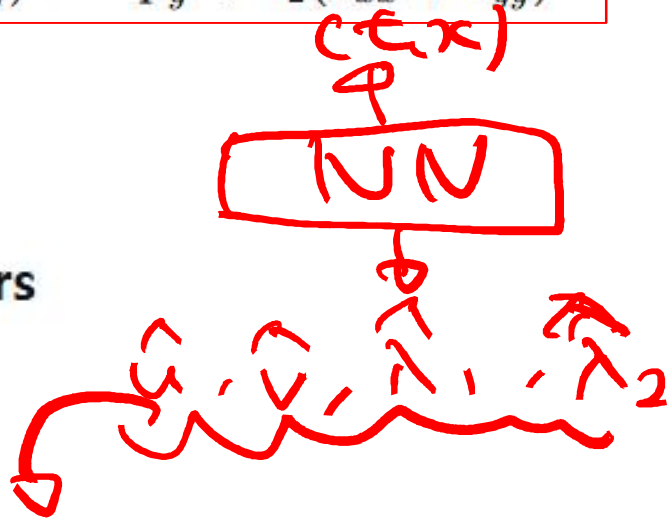
Example 1/3

- **2D Navier–Stokes Equation**
  - Navier–Stokes equations describe the physics of many phenomena : fluid flow

$$u_t + \lambda_1(uu_x + vu_y) = -p_x + \lambda_2(u_{xx} + u_{yy})$$

$$v_t + \lambda_1(uv_x + vv_y) = -p_y + \lambda_2(v_{xx} + v_{yy})$$

- u, v: velocity components
- p: pressure
- λ_1, λ_2: **unknown parameters**



- **PDE residuals**

$$f := u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy}),$$
$$g := v_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy}),$$

Example 2/3

- **2D Navier–Stokes Equation**
  - Loss function

$$\text{MSE} = \underbrace{\text{velocity data loss}}_{\text{fit observations}} + \underbrace{\text{PDE residual loss}}_{\text{enforce physics}}$$

$$MSE := \frac{1}{N}\sum_{i=1}^{N}\left(|u(t^i,x^i,y^i)-u^i|^2+|v(t^i,x^i,y^i)-v^i|^2\right)+\frac{1}{N}\sum_{i=1}^{N}\left(|f(t^i,x^i,y^i)|^2+|g(t^i,x^i,y^i)|^2\right).$$

Example 3/3

- **2D Navier–Stokes Equation**
  - λ_1: error ≈ 0.078%, λ_2: error ≈ 4.67%

